K-SVDD Control Chart

Algorithm Technical Documentation

K-SVDD Control Chart for Multivariate Process Monitoring

Language: Python
Algorithm: K-SVDD Control Chart
Date: March 2026

1. Overview

This document describes the algorithmic implementation of the K-SVDD Control Chart, a multivariate statistical process monitoring method. The algorithm fits a Support Vector Data Description (SVDD) to a representative reference set selected from historical training data and uses the resulting hypersphere radius as a fixed control limit. Incoming test observations are scored by their distance from the SVDD hypersphere centre, and an alarm is raised whenever this distance exceeds the radius.

The reference set is selected through a two-stage screening procedure: a preliminary SVDD is first fitted on the full training data to compute per-observation distances, and observations falling in the 35th-to-50th percentile band of these distances are retained as candidates. A random subsample of 100 observations is then drawn from this candidate pool to form the final reference set. The monitoring SVDD is fitted on this reference set, and no further updates occur during online monitoring.

Module	Role
Data ingestion and normalisation	Load train/test CSVs; apply joint RobustScaler normalisation
Screening SVDD	Fit SVDD on the full training set; rank observations by distance from the hypersphere centre
Reference set construction	Select observations from the 35th–50th percentile distance band; random subsample to S0 = 100
Monitoring SVDD	Fit a second SVDD on the reference set; extract radius as the fixed control limit
Static online scoring	Compute SVDD distance for every test observation; compare to

Module	Role
	fixed radius
Alarm detection and visualisation	Count and index alarm events; plot distance trajectory against control limit

2. Data Ingestion and Normalisation

2.1 Input Files

The pipeline requires two CSV files: `real example-testdata.csv` and `real example-traindata.csv`. The first column of each file is dropped via `.iloc[0:n, 1:]`, treating it as a row-index column. The column `Confirmedcases` is separately extracted from the training file as the one-dimensional array `R`. Unlike the Time-Adaptive SVDD and D-SVM pipelines, `R` is not used in the reference set selection step of this algorithm; it is loaded but plays no operational role in the K-SVDD procedure.

Variable	Source	Shape	Role in K-SVDD
`y_test`	`real example-testdata.csv`	(n_test, p)	Observations to be monitored; scored against the fitted hypersphere
`y_train`	`real example-traindata.csv`	(n_train, p)	Historical reference data; used for screening SVDD and reference set construction
`R`	Column Confirmedcases of traindata	(n_train,)	Loaded but not used in the monitoring procedure

2.2 Joint Robust Normalisation

Training and test data are concatenated before fitting the scaler so that a single transformation is applied consistently to both sets:

```
All = np.r_[y_test, y_train]
robust = preprocessing.RobustScaler()
All = robust.fit_transform(All)
y_test  = All[:y_test.shape[0], :]
y_train = All[y_test.shape[0]:, :]
```

`RobustScaler` centres each feature by its median and scales by its interquartile range. Fitting on the combined array prevents the scaling parameters from being determined by training data alone, maintaining consistency across both sets.

3. Screening SVDD and Distance-Based Candidate Selection

3.1 Screening SVDD Fit

A preliminary SVDD is fitted on the full normalised training set `y_train` using an RBF kernel. Every training observation is then scored by its distance from the fitted hypersphere centre:

```
sample_ids = np.arange(0, num_trains)              # integer index for each training
row
svdd = BaseSVDD(C=1, gamma=0.4, kernel='rbf', display='off')
svdd.fit(y_train)
distances = svdd.get_distance(y_train)             # shape: (n_train,)
distance_dict = dict(zip(sample_ids, distances))   # {row_index: distance}
```

The screening SVDD uses `C = 1`, which in SVDD terminology sets the upper bound on the fraction of training observations allowed outside the hypersphere. With `C = 1` the solver attempts to enclose all training observations, producing a conservative boundary that faithfully reflects the full training distribution.

3.2 Distance-Band Selection

The (index, distance) pairs are sorted in ascending order of distance. Observations whose sorted position falls between the 35th and 50th percentiles of the full ranking are selected as candidates:

```
sorted_items = sorted(distance_dict.items(), key=lambda x: x[1])
n = len(sorted_items)
q1_index = int(n * 0.35)    # lower rank bound (35th percentile of sorted list)
q3_index = int(n * 0.50)    # upper rank bound (50th percentile of sorted list)
selected_sample_ids = [item[0] for item in sorted_items[q1_index:q3_index]]
valid_indices = [idx for idx in selected_sample_ids if 0 <= idx < y_train.shape[0]]
y_train_pro = y_train[valid_indices, :]
```

The selection targets observations at moderate distance from the screening SVDD centre: they are neither the closest observations (which may cluster near degenerate support vectors) nor the most outlier-like (which would distort the monitoring SVDD). The `valid_indices` filter is a safety check that discards any out-of-range indices before indexing `y_train`; under normal operation all selected indices are valid.

Note: This selection is based entirely on SVDD distances computed from the screening model, not on any univariate feature column such as `R`. This is the principal structural difference between the K-SVDD reference set construction and those of the Time-Adaptive SVDD and D-SVM pipelines, which apply quantile thresholds directly to the `R` column.

4. Reference Set Construction

From the candidate pool `y_train_pro`, a reference set of `S0 = 100` observations is drawn uniformly at random without replacement:

```
S0 = 100
S0_0 = y_train_pro.shape[0]                          # size of candidate pool
row_indices = np.random.choice(S0_0, size=S0, replace=False)
y_S0 = y_train[row_indices, :]                       # shape: (100, p)
```

Note: The random draw uses `row_indices` to index `y_train` (the full overwritten training array), not `y_train_pro`. Because `row_indices` are positions within the candidate pool of size `S0_0`, and `y_train_pro = y_train[valid_indices, :]`, this indexing is consistent provided `S0_0 ≤ y_train.shape[0]`, which holds by construction.

The reference set `y_S0` is fixed after construction. No rolling update, frozen-set mechanism, or online growth is applied during monitoring. This static design makes K-SVDD the simplest of the SVDD-based methods considered: the monitoring SVDD is fitted once and never revisited.

5. Monitoring SVDD and Control Limit

5.1 SVDD Fit on Reference Set

A second SVDD is fitted exclusively on the 100-row reference set `y_S0`. This monitoring SVDD uses different hyperparameters from the screening SVDD:

```
svdd = BaseSVDD(C=0.4, gamma=0.03, kernel='rbf', display='off')
svdd.fit(y_S0)
```

Parameter	Screening SVDD	Monitoring SVDD	Effect of difference
C	1	0.4	Monitoring SVDD allows up to 40% of reference observations outside the hypersphere, producing a looser boundary
gamma	0.4	0.03	Much wider RBF kernel in the monitoring SVDD; the hypersphere is smoother and less sensitive to individual observations

5.2 Hypersphere Radius as Fixed Control Limit

The control limit is taken directly as the hypersphere radius returned by the fitted monitoring SVDD:

```
radius = svdd.radius   # scalar: the fitted hypersphere radius
```

The radius is the distance from the hypersphere centre to the support vectors in the feature space induced by the RBF kernel. It is a scalar constant determined entirely by the reference set and the SVDD hyperparameters. No bootstrap calibration, quantile estimation, or outer loop is required: the control limit is an analytic byproduct of the SVDD optimisation.

This is the most direct control limit derivation of all four methods considered. The Time-Adaptive SVDD uses a 1,000-replicate data-space bootstrap; the D-SVM uses a 100-outer-replicate nested bootstrap; and the RTSVDD uses a per-step quantile of reference distances. K-SVDD uses the hypersphere radius itself, requiring no additional computation after the SVDD fit.

6. Static Online Scoring

The monitoring phase is a single vectorised operation rather than an iterative loop. All `n_test` test observations are scored simultaneously by computing their distances from the monitoring SVDD hypersphere centre:

```
X = y_test
distance = svdd.get_distance(X)          # shape: (n_test, 1)
n = distance.shape[0]
radius_f = np.ones((n, 1)) * radius      # control limit broadcast to (n_test, 1)
```

The distance matrix `distance` contains one entry per test observation. Each entry is the distance of that observation from the SVDD hypersphere centre in the RBF kernel-induced feature space. Observations inside the hypersphere have distance less than `radius`; those outside have distance greater than `radius`.

This batch scoring approach contrasts with the step-by-step loop used in the RTSVDD and D-SVM pipelines. There is no per-step model refit, no reference set update, and no window construction. The monitoring SVDD fitted in Section 5 is applied directly to the full test matrix in one call.

6.1 Alarm Detection

Alarms are identified by comparing each test observation's distance to the fixed radius in two equivalent ways:

```
# Method 1: element-wise loop (prints n_alarm count)
n_alarm = 0
for i in range(n):
    if radius < distance[i][0]:
        n_alarm += 1

# Method 2: vectorised index extraction
n_alarm_indicate = np.where(distance > radius_f)[0][:]
```

Both methods produce equivalent results. The loop computes the scalar count `n_alarm`, while `np.where` returns the array of step indices where the distance exceeds the radius. An observation triggers an alarm if and only if its SVDD distance strictly exceeds the hypersphere radius:

$$\text{Alarm at step } i \iff \text{distance}(x_i) > \text{radius}$$

7. Output Collection

Output Variable	Shape	Content
`distance`	(n_test, 1)	SVDD distance from hypersphere centre for each test observation
`radius_f`	(n_test, 1)	Fixed control limit (hypersphere radius) broadcast to full test length
`n_alarm`	scalar	Total count of alarm events (distance > radius)
`n_alarm_indicate`	(n_alarm,)	Array of step indices where distance > radius_f

Both `n_alarm` and `n_alarm_indicate` are printed to console. The wall-clock runtime of the full pipeline is also reported via the `time` module.

8. Visualisation

The function `plot_distance(radius, distance)` renders the SVDD distance trajectory against the fixed control limit as a time-series chart.

Element	Style	Represents
Control limit	Red solid line, linewidth 3	`radius_f`
Distance statistic	Black dotted line with circular markers	`distance`

The x-axis tick labels map integer indices `[0, 49, 99, 149, 199, 249, 299]` to the date strings `['1-Mar', '20-Apr', '9-Jun', '29-Jul', '17-Sept', '6-Nov', '26-Dec']`. These labels are hardcoded to the reference dataset and should be adjusted for other time series.

9. Hyperparameter Summary

Parameter	Value	Where Used	Role
C (screening SVDD)	1	First SVDD on full y_train	Allows no outliers; conservative boundary enclosing all training observations
gamma (screening SVDD)	0.4	First SVDD on full y_train	RBF kernel width for screening
q1_index percentile	0.35	Distance-band selection	Lower rank bound of the

Parameter	Value	Where Used	Role
			candidate selection band
q3_index percentile	0.50	Distance-band selection	Upper rank bound of the candidate selection band
S0	100	Reference set size	Number of observations drawn without replacement from the candidate pool
C (monitoring SVDD)	0.4	Second SVDD on y_S0	Allows up to 40% of reference observations outside the hypersphere
gamma (monitoring SVDD)	0.03	Second SVDD on y_S0	Wide RBF kernel; produces a smoother, more generalised hypersphere boundary

10. Dependencies and Execution

10.1 Required Packages

Package	Usage
numpy	Array operations, random sampling
pandas	CSV ingestion
src.BaseSVDD	SVDD fitting, radius retrieval, and distance computation (custom/local module)
sklearn.preprocessing.RobustScaler	Joint normalisation of train and test data
matplotlib / pylab	Control chart visualisation; custom x-axis tick labels
time	Wall-clock timing of the full pipeline

10.2 Execution

Place `real example-traindata.csv` and `real example-testdata.csv` in the working directory, ensure `src/BaseSVDD.py` is accessible via `sys.path`, and execute:

```
python KSVDD.py
```

References

[1] Tax, D. M. J., & Duin, R. P. W. (2004). Support vector data description. *Machine Learning*, 54(1), 45–66.

[2] Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.

[3] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[4] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.